

Module 7: Functions

7.1 Introduction to Functions

- A function is a named block of code that performs a specific task
- A function can be written once and called (used) as many times as needed from anywhere in the program
- Benefits of functions
 - Reusability: write once, use many times
 - Readability: main becomes shorter and easier to read
 - Modularity: divides a large problem into smaller manageable pieces
 - Easier debugging: problems can be isolated to specific functions
- Types of functions in C++
 - Built-in (library) functions: provided by C++ standard library, example: `sqrt()`, `abs()`, `toupper()`
 - User-defined functions: written by the programmer

7.2 Defining a Function

- A function definition has four parts
 - Return type: the data type of the value the function sends back (use `void` if nothing is returned)
 - Function name: follows the same naming rules as variable names
 - Parameter list: input values the function needs, listed inside parentheses (can be empty)
 - Function body: the code inside braces that executes when the function is called
- Syntax

```
return_type functionName(parameter1_type parameter1_name, ...) {  
    // function body  
    return value; // if return type is not void  
}
```
- Example: a function that adds two integers and returns the result

```
int add(int a, int b) {  
    int result = a + b;  
    return result;  
}
```

7.3 Calling a Function

- A function is invoked (called) by writing its name followed by arguments in parentheses
- The program jumps to the function, executes it, then returns to where it was called
- Example of calling the `add` function

```
int sum = add(5, 3); // sum gets 8  
cout << add(10, 20); // prints 30
```
- A `void` function is called as a statement, not used in an expression

```
printMessage(); // correct
int x = printMessage(); // error: void function returns nothing
```

7.4 Function Prototypes

- By default, a function must be defined before it is called
- If `main()` is at the top and helper functions are below it, the compiler does not know about them yet
- A function prototype declares the function signature (name, parameters, return type) to the compiler before the actual definition

- Syntax: `return_type functionName(parameter_types);`

- Example

```
int add(int, int); // prototype at the top
int main() {
    cout << add(5, 3); // allowed because prototype was seen
}

int add(int a, int b) { return a + b; } // definition at the bottom
```

7.5 Parameters and Arguments

- Formal parameters: variable names listed in the function definition
- Actual arguments: the real values passed when the function is called
- The number and types of arguments must match the parameters exactly
- Functions with no parameters: use empty parentheses or void

```
void greet() { cout << "Hello!"; }
greet();
```

- Functions with multiple parameters: listed and separated by commas
- The order of arguments must match the order of parameters

7.6 Return Values

- A function with a non-void return type must use the return statement
- return expression; sends the value back to the caller
- The function exits immediately when return is reached
- A void function can use return; (with no value) to exit early if needed
- Using the returned value
- In assignment: `int result = square(5);`
- In an expression: `cout << square(5) * 2;`
- As an argument: `cout << add(square(2), square(3));`

7.7 Pass by Value

- When a variable is passed to a function, a copy of the value is made
- The function works with the copy, not the original
- Any changes to the parameter inside the function do not affect the original variable

- Example

```
void addTen(int x) { x += 10; } // x is a copy
int num = 5;
addTen(num);
cout << num; // still 5, not 15
```

7.8 Pass by Reference

- Using & in the parameter declaration, the function receives the actual variable, not a copy
- Changes to the parameter inside the function directly affect the original variable
- Example

```
void addTen(int &x) { x += 10; }
int num = 5;
addTen(num);
cout << num; // now 15
```

- Use pass by reference when you want to modify the original variable or to avoid copying large data structures

7.9 Default Arguments

- A parameter can be given a default value in the function definition or prototype
- If the caller does not provide that argument, the default is used
- Default arguments must be listed from right to left (rightmost first)

```
void greet(string name, string msg = "Hello") {
    cout << msg << ", " << name << "!";
}
greet("Ali");           // prints: Hello, Ali!
greet("Sara", "Hi");    // prints: Hi, Sara!
```

7.10 Function Overloading

- C++ allows multiple functions with the same name as long as their parameter lists differ
- The compiler picks the right version based on the types and number of arguments
- Overloading by number of parameters

```
int area(int side);           // square
int area(int length, int width); // rectangle
```
- Overloading by type of parameters

```
double divide(double a, double b);
int divide(int a, int b);
```
- Return type alone does not distinguish overloaded functions

7.11 Recursion

- A function that calls itself is called recursive
- Every recursive function needs two things

Base case: a condition that stops the recursion (no self-call)

Recursive case: the part where the function calls itself with a simpler version of the problem

- Without a base case, the function calls itself forever and causes a stack overflow
- Factorial example

```
int factorial(int n) {
    if (n == 0 || n == 1) return 1; // base case
    return n * factorial(n - 1);    // recursive case
}
```

- How the call stack works during recursion

Each function call is pushed onto the call stack

When the base case is reached, functions start returning and popping off the stack

- Recursion vs iteration

Any problem solvable with recursion can also be solved with a loop

Recursion is often more elegant but uses more memory (stack frames)

Iteration is usually faster and safer for problems with deep recursion

7.12 Scope of Variables

- Local variables
- Declared inside a function or a block
- Only accessible within that function or block
- Created when the block starts and destroyed when it ends
- Two local variables in different functions can have the same name without conflict
- Global variables
- Declared outside all functions, at the top of the file
- Accessible from any function in the file
- Persist for the entire duration of the program
- Should be used sparingly because they can be modified from anywhere, making debugging harder
- Variable shadowing
- A local variable with the same name as a global variable hides the global one within its scope
- The local variable takes priority

7.13 Storage Classes

- auto: the default for all local variables; created on the stack and destroyed when scope ends
- static: a static local variable retains its value between function calls; initialized only once

```
void countCalls() {
    static int count = 0;
    count++;
}
```

```
cout << "Called " << count << " times" << endl;  
}
```

- `extern`: used to declare a variable that is defined in another source file
- `register`: hints to the compiler to store the variable in a CPU register for speed (mostly ignored in modern compilers)